

# Lab 0

Date

No.

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 100
int count = 0; node * top = NULL;
typedef struct st {
    int operand;
    struct st * below;
} node;

node * push(node tr)
{
    node * ptr;
    ptr = (node *) malloc(sizeof(node));
    if(count >= SIZE) return NULL;
    *ptr = tr;
    ptr->below = top;
    top = ptr;
    count++;
    return ptr;
}

node * pop()
{
    static node popped_node;
    if(count <= 0) return NULL;
    node * ptr;
    popped_node = *top;
    ptr = top;
    top = top->below;
    free(ptr);
    count--;
    return &popped_node;
}

int main()
{
    node t, *ptr;
    int ans, result;

    int operand1, operand2;
    FILE * indata;
    char this1;
    indata = fopen("postfix.inf", "r");
    if(indata == NULL)
    {
        printf("Input file not found");
        return 0;
    }
    do {
        fscanf(indata, "%c", &this1);
        if(this1 != '=') printf("%c", this1);
        else if(this1 != 'x') printf("%c", this1);
        if(this1 >='1' && this1 <='9') ans = 1;
        else if(this1 == '+' || this1 == '-' || this1 == '*' || this1 == '/') ans = 2;
        else if(this1 == '=') ans = 3;
        else if(this1 == 'x') ans = 4;

        switch(ans)
        {
            case 1:
                t.operand = this1 - 48;
                if(push(t) == NULL)
                    printf("STACK FULL"); break;
            case 2:
                if((ptr = pop()) != NULL)
                    operand2 = ptr->operand;
                else printf("STACK EMPTY");
                if((ptr = pop()) != NULL)
                    operand1 = ptr->operand;
                else printf("STACK EMPTY");
            case 3:
                switch(this1)
                {
                    case '+':
                        result = operand1 + operand2;
                        break;
                    case '-':
                        result = operand1 - operand2;
                        break;
                    case '*':
                        result = operand1 * operand2;
                        break;
                    case '/':
                        result = operand1 / operand2;
                        break;
                }
            case 4:
                printf("Invalid input");
                break;
        }
    } while(fgetc(indata) != '\n');
```

A'ZONE



# Lab 2

Date

No.

```
#include <stdio.h>
int main() {
    int count;
    FILE * indata, * outdata;
    char this1;
    unsigned packed, temp, mask;
    indata = fopen("d:\\digit.inp", "rt");
    outdata = fopen("d:\\digit.out", "wb");
    if (indata == NULL || outdata == NULL) {
        printf("In File error 1");
        fclose(outdata);
    }
    printf("\n\n Original Text:");
    count = 0;
    packed = 0;
    while (fscanf(indata, "%c", &this1) == 1) {
        printf("%c", this1);
        this1 -= '0';
        temp = (unsigned) this1;
        temp <<= 32 - 4 * (count + 1);
        packed |= temp;
        count++;
        if (count == 8) {
            fwrite(&packed, sizeof(unsigned), 1, outdata);
            count = 0;
            packed = 0;
        }
    }
    fclose(indata);
    printf("\n Decompressed Text:");
    while (fread(&packed, sizeof(unsigned), 1, indata) == 1) {
        mask = 15 << 8;
        for (count = 0; count < 8; count++) {
            temp = packed & mask;
            this1 = (char) (temp >> 32 - 4 * (count + 1));
        }
    }
}
```

左边移出的位: 直接丢弃  
右边: 补0  
是 unsigned  
完全就是按位截断  
不存在未定义行为

A'ZONE